

- 软件工程课程实验4：使用 Git 管理项目
 - 1 实验目标与过程
 - 2 安装 Git
 - 2.1 本地机器上安装 Git
 - 2.2 申请 GitHub 帐号
 - 3 Git 操作过程
 - 3.1 实验场景 1：仓库创建与提交
 - R0 查看状态
 - R1 本地初始化 Git 仓库并纳入 Git 管理
 - R2 提交
 - 手工修改 3 个文件
 - R3 查看上次提交后的文件修改和具体差异
 - R4 重新提交
 - 再次修改同样 3 个文件
 - R5 重新提交
 - R6 撤销最后一次提交
 - R7 查询提交记录
 - 3.2 实验场景 2：分支管理
 - R8 从 GitHub URL 克隆已有仓库到本地
 - R9 获得该仓库的全部分支
 - R10 在 B2 分支基础上创建新分支 C4
 - R11 在 C4 上修改 4 个文件并提交
 - R12 在 B3 分支上对同样 4 个文件做不同修改并提交
 - R13 将 C4 和 B3 分支合并，若有冲突则手工消解
 - R14 查看哪些分支已经合并、哪些分支尚未合并
 - R15 删除合并后的分支，并将尚未合并的分支合并到学号分支
 - 3.3 实验场景 3：远程分支管理
 - R16 将本地学号分支推送到 GitHub
 - R17 将 R1 到 R7 的结果推送到 GitHub
 - R18 在 GitHub 网站查看两个仓库当前状态
 - 4 小结
 - 4.1 使用 Git 的优点
 - 4.2 Git 在个人开发和团队开发中的差异
 - 4.3 与其他版本控制系统的比较
 - 4.4 适合与不适合使用 Git 的情况
 - 4.5 其他思考

软件工程课程实验4：使用 Git 管理项目

1 实验目标与过程

本次实验目标是学习使用 Git 管理项目版本，掌握仓库初始化、文件暂存、提交、查看差异、撤销提交、分支创建、分支合并、冲突解决以及远程分支推送等基本操作。

实验过程分为三个场景：

1. 在本地创建源码项目，初始化 Git 仓库，完成多次提交并撤销最后一次提交。
2. 以已有远程仓库为基础，获取分支、创建新分支、修改同一批文件、合并分支并解决冲突。
3. 将本地分支和本地仓库提交结果推送到远程 GitHub 仓库。

本工作区内已准备的实验目录如下：

- `scene1-local-repo`：场景 1 本地仓库。
- `scene1-origin.git`：场景 1 的本地远程源模拟仓库。
- `scene2-seed-project`：场景 2 初始仓库，用于构造 B1、B2、B3。
- `scene2-origin.git`：场景 2 的本地远程源模拟仓库。
- `scene2-clone`：场景 2 克隆后执行 R8 到 R16 的仓库。

说明：当前环境没有 GitHub CLI 或浏览器登录凭据，因此 GitHub 远程仓库已用本地裸仓库模拟。正式提交实验时，需要把命令中的本地远程地址替换成自己的 GitHub 仓库 URL，并补充网页截图。

2 安装 Git

2.1 本地机器上安装 Git

执行命令：

```
git --version
```

本机 Git 版本：

```
git version 2.50.1.windows.1
```

已配置的 Git 用户信息：

```
git config --global user.name  
git config --global user.email
```

输出：

```
xjtatnku  
2691093659@qq.com
```

截图位置：粘贴 `git --version` 和 Git Bash / PowerShell 运行界面截图。

2.2 申请 GitHub 帐号

GitHub 帐号名称：xjtatnku（请按实际 GitHub 帐号核对）。

本次实验涉及的 GitHub 项目 URL：

- 场景 1 仓库：<https://github.com/xjtatnku/git-lab-local-project>（请替换成实际仓库 URL）
- 场景 2 仓库：<https://github.com/xjtatnku/branch-lab-project>（请替换成实际仓库 URL）

截图位置：

- 粘贴 GitHub 账号信息页面截图。
- 粘贴场景 1 项目页面截图。
- 粘贴场景 2 项目页面截图。

3 Git 操作过程

3.1 实验场景 1：仓库创建与提交

R0 查看状态

在每次 Git 操作前后，使用如下命令查看工作区、暂存区和仓库状态：

```
git status --short --branch
```

初始状态输出示例：

```
## No commits yet on master
?? README.md
?? docs/
?? src/
?? tests/
```

截图位置：粘贴 `git status --short --branch` 的命令与结果。

R1 本地初始化 Git 仓库并纳入 Git 管理

命令：

```
cd .\scene1-local-repo
git init
git add .
git status --short --branch
```

暂存区状态输出示例：

```
## No commits yet on master
A  README.md
A  docs/design.md
A  src/app.py
A  src/calculator.py
A  src/utils.py
A  tests/test_calculator.py
```

截图位置：粘贴初始化仓库、加入暂存区和查看状态的命令结果。

R2 提交

命令：

```
git commit -m "Initial local project"
```

输出摘要：

```
[master (root-commit) 8ea6cd2] Initial local project  
6 files changed, 48 insertions(+)
```

截图位置：粘贴第一次提交结果。

手工修改 3 个文件

修改文件：

- `README.md`
- `src/app.py`
- `src/calculator.py`

主要修改内容：

- 在 `README.md` 增加用法说明。
- 在 `src/calculator.py` 增加 `subtract` 函数。
- 在 `src/app.py` 调用并输出减法结果。

R3 查看上次提交后的文件修改和具体差异

命令：

```
git status --short --branch  
git diff -- README.md src/app.py src/calculator.py
```

差异统计：

```
README.md      | 8 +++++++  
src/app.py     | 4 ++--  
src/calculator.py | 5 +++-  
3 files changed, 14 insertions(+), 3 deletions(-)
```

若需要查看暂存区与最后一次提交的差异，先执行：

```
git add README.md src/app.py src/calculator.py
git diff --cached --stat
```

截图位置：粘贴 `git status`、`git diff`、`git diff --cached` 的命令与结果。

R4 重新提交

命令：

```
git add README.md src/app.py src/calculator.py
git commit -m "Add subtraction demo"
```

输出摘要：

```
[master 02c9248] Add subtraction demo
3 files changed, 14 insertions(+), 3 deletions(-)
```

截图位置：粘贴第二次提交结果。

再次修改同样 3 个文件

修改文件：

- `README.md`
- `src/app.py`
- `src/calculator.py`

主要修改内容：

- 增加除法说明。
- 增加 `divide` 函数。
- 在程序入口输出除法结果。

R5 重新提交

命令：

```
git status --short --branch
git diff -- README.md src/app.py src/calculator.py
git add README.md src/app.py src/calculator.py
git commit -m "Add division demo"
```

输出摘要：

```
[master dd8f5a6] Add division demo
3 files changed, 11 insertions(+), 1 deletion(-)
```

截图位置：粘贴再次修改、查看差异和第三次提交结果。

R6 撤销最后一次提交

本次使用 `git revert` 撤销最后一次提交。该方法会生成一条新的撤销提交，适合保留历史记录。

命令：

```
git revert HEAD --no-edit
```

输出摘要：

```
[master b44732a] Revert "Add division demo"
3 files changed, 1 insertion(+), 11 deletions(-)
```

说明：如果实验要求“删除最后一次提交”而不是“生成反向提交”，也可以使用：

```
git reset --hard HEAD~1
```

但该命令会移动分支指针并丢弃工作区修改，使用前需要确认。

截图位置：粘贴撤销最后一次提交的命令与结果。

R7 查询提交记录

命令：

```
git log --oneline --decorate --graph --all
```

输出：

```
* b44732a (HEAD -> master) Revert "Add division demo"  
* dd8f5a6 Add division demo  
* 02c9248 Add subtraction demo  
* 8ea6cd2 Initial local project
```

截图位置：粘贴提交记录查询结果。

3.2 实验场景 2：分支管理

实验要求中，初始 B1、B2、B3 需要通过 GitHub Web 页面建立。本工作区中已用 `scene2-seed-project` 构造了等价仓库：

- 初始分支：B1
- 并行分支：B2、B3
- 初始项目文件数量：12 个

GitHub Web 页面操作记录：

1. 在 GitHub 新建仓库 `branch-lab-project`。
2. 在默认分支上传不少于 10 个文件，形成初始分支 B1。
3. 从 B1 创建分支 B2，修改部分文件并提交。
4. 从 B1 创建分支 B3，修改部分文件并提交。

截图位置：粘贴 GitHub 上 B1、B2、B3 分支和提交记录页面。

R8 从 GitHub URL 克隆已有仓库到本地

正式 GitHub 命令：

```
git clone https://github.com/xjtatnku/branch-lab-project.git scene2-clone
```

本地模拟实际执行命令：


```
git clone .\scene2-origin.git scene2-clone
```

输出摘要：

```
Cloning into 'scene2-clone'...  
done.
```

截图位置：粘贴克隆命令与结果。

R9 获得该仓库的全部分支

命令：

```
cd .\scene2-clone  
git fetch --all --prune  
git branch -a
```

输出：

```
* B1  
remotes/origin/B1  
remotes/origin/B2  
remotes/origin/B3  
remotes/origin/HEAD -> origin/B1
```

截图位置：粘贴获取分支和查看分支列表的命令结果。

R10 在 B2 分支基础上创建新分支 C4

命令：

```
git switch -c B2 origin/B2  
git switch -c C4
```

输出摘要：

```
branch 'B2' set up to track 'origin/B2'.  
Switched to a new branch 'B2'  
Switched to a new branch 'C4'
```

截图位置：粘贴创建 B2 本地跟踪分支和 C4 分支的命令结果。

R11 在 C4 上修改 4 个文件并提交

修改文件：

- README.md
- src/report.py
- docs/test-plan.md
- data/sample.json

命令：

```
git status --short --branch  
git diff --stat  
git add README.md src/report.py docs/test-plan.md data/sample.json  
git commit -m "Apply C4 summary updates"
```

输出摘要：

```
[C4 3ee0109] Apply C4 summary updates  
4 files changed, 7 insertions(+), 5 deletions(-)
```

截图位置：粘贴 C4 修改、查看状态、提交结果。

R12 在 B3 分支上对同样 4 个文件做不同修改并提交

命令：

```
git switch -c B3 origin/B3  
git status --short --branch  
git diff --stat  
git add README.md src/report.py docs/test-plan.md data/sample.json  
git commit -m "Apply B3 report updates"
```

输出摘要：

```
[B3 ab6bb39] Apply B3 report updates
4 files changed, 9 insertions(+), 7 deletions(-)
```

截图位置：粘贴 B3 修改同样 4 个文件并提交的结果。

R13 将 C4 和 B3 分支合并，若有冲突则手工消解

当前在 B3 分支，执行：

```
git merge C4
```

冲突输出：

```
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Auto-merging data/sample.json
CONFLICT (content): Merge conflict in data/sample.json
Auto-merging docs/test-plan.md
CONFLICT (content): Merge conflict in docs/test-plan.md
Auto-merging src/report.py
CONFLICT (content): Merge conflict in src/report.py
Automatic merge failed; fix conflicts and then commit the result.
```

查看冲突文件：

```
git status --short --branch
git diff --name-only --diff-filter=U
```

冲突文件：

```
README.md
data/sample.json
docs/test-plan.md
src/report.py
```

手工编辑冲突文件，删除 <<<<<<、=====、>>>>>> 标记，合并两边内容后执行：

```
git add README.md data/sample.json docs/test-plan.md src/report.py
git status --short --branch
git commit -m "Merge C4 into B3"
```

输出摘要：

```
[B3 49958ec] Merge C4 into B3
```

截图位置：粘贴合并冲突、冲突文件列表、解决冲突、提交合并结果。

R14 查看哪些分支已经合并、哪些分支尚未合并

命令：

```
git branch --merged
git branch --no-merged
git log --oneline --decorate --graph --all --max-count=20
```

`git branch --merged` 输出：

```
B1
B2
* B3
C4
```

`git branch --no-merged` 输出为空，表示当前没有尚未合并到当前分支的本地分支。

截图位置：粘贴已合并分支、未合并分支和提交图。

R15 删除合并后的分支，并将尚未合并的分支合并到学号分支

本次合并后 `C4` 已经合并到 `B3`，因此删除 `C4`：

```
git branch -d C4
```

输出：

```
Deleted branch C4 (was 3ee0109).
```

由于没有尚未合并的本地分支，创建学号分支并确认合并状态。本地实验中使用 2024000000 作为学号占位，请替换为你的真实学号：

```
git switch -c 2024000000
git merge B2
git branch --merged
git branch --no-merged
```

输出摘要：

```
Switched to a new branch '2024000000'
Already up to date.
```

如果需要改成真实学号分支名，执行：

```
git branch -m 2024000000 你的真实学号
```

截图位置：粘贴删除 C4、创建学号分支和查看合并状态的结果。

3.3 实验场景 3：远程分支管理

R16 将本地学号分支推送到 GitHub

正式 GitHub 命令：

```
git remote -v
git push -u origin 你的真实学号
```

本地模拟实际执行命令：

```
git push -u origin 2024000000
```

输出摘要：

```
branch '2024000000' set up to track 'origin/2024000000'.
* [new branch]      2024000000 -> 2024000000
```

截图位置：粘贴推送学号分支结果。

R17 将 R1 到 R7 的结果推送到 GitHub

正式 GitHub 命令：

```
cd ../scene1-local-repo
git remote add origin https://github.com/xjtatnku/git-lab-local-project.git
git push -u origin master
```

本地模拟实际执行命令：

```
git remote add origin ../scene1-origin.git
git push -u origin master
```

输出摘要：

```
branch 'master' set up to track 'origin/master'.
* [new branch]      master -> master
```

截图位置：粘贴推送场景 1 仓库结果。

R18 在 GitHub 网站查看两个仓库当前状态

需要在 GitHub 网页查看：

- 场景 1 仓库提交记录是否包含：
 - Initial local project
 - Add subtraction demo
 - Add division demo
 - Revert "Add division demo"
- 场景 2 仓库是否包含：
 - B1

- B2
- B3
- 学号分支
- 合并提交 Merge C4 into B3

截图位置：

- 粘贴场景 1 GitHub 仓库首页、提交记录页面。
- 粘贴场景 2 GitHub 仓库分支列表、学号分支页面、提交记录页面。

4 小结

4.1 使用 Git 的优点

与直接复制文件夹或手工保存多个版本相比，Git 能准确记录每次修改的作者、时间、提交说明和具体差异。出现错误时，可以通过 `git log`、`git diff`、`git revert` 等命令定位问题并恢复到可用状态。Git 还支持分支开发，使不同功能可以并行推进，不影响主线代码。

4.2 Git 在个人开发和团队开发中的差异

个人开发中，Git 主要用于版本备份、回滚和保存阶段性成果。团队开发中，Git 还承担协作管理作用，例如通过分支隔离任务、通过 Pull Request 进行代码审查、通过合并记录追踪多人修改，并在冲突发生时帮助开发者定位和解决差异。

4.3 与其他版本控制系统的比较

常见版本控制系统包括 CVS、SVN 和 Git。CVS 和 SVN 更偏集中式管理，历史记录主要存放在中央服务器上，离线能力较弱。Git 是分布式版本控制系统，每个本地仓库都保存完整历史记录，因此可以离线提交、查看日志、创建分支和回滚。Git 的分支创建与合并成本较低，更适合频繁迭代和多人并行开发。

4.4 适合与不适合使用 Git 的情况

适合使用 Git 的情况：

- 程序代码、实验文档、配置文件等需要持续修改和追踪历史。
- 多人协作开发同一个项目。
- 需要保留多个功能分支或实验分支。
- 需要在出错后快速回滚或定位修改来源。

没必要使用 Git 的情况：

- 一次性临时文件，没有长期维护价值。
- 超大二进制文件为主且没有配置 Git LFS。
- 修改过程不需要历史记录，也没有协作需求。

4.5 其他思考

本次实验中，分支合并冲突是理解 Git 的关键环节。冲突并不表示 Git 出错，而是说明两个分支修改了同一位置，Git 无法自动判断应该保留哪一边。解决冲突时需要结合业务含义，而不是简单保留某一边内容。提交信息也很重要，清晰的提交信息可以让后续查看历史和排查问题更高效。